

Unidad 01 – Introducción a la Informática

Cátedra: Programación en Computación – UTN FRRQ **Carrera:** Ingeniería Electromecánica – 2.º año **Docentes:** Longhi Pablo, Talijancic Iván

Por qué arrancamos por acá

Vas a pasar el cuatrimestre escribiendo programas. Un programa no es magia: es una lista de instrucciones que **una máquina física ejecuta**, y esa máquina tiene partes concretas con límites concretos.

Entender qué hay abajo te va a servir para responder preguntas que aparecen todo el tiempo:

- ¿Por qué mi programa "se colgó"?
- ¿Por qué al cerrar el programa se perdieron los datos?
- ¿Por qué un programa que anda en mi máquina no anda en la de al lado?
- ¿Qué diferencia hay entre *guardar* y *tener abierto*?

Ninguna de esas se contesta mirando el código. Se contestan sabiendo **dónde está cada cosa**.

El curso de un vistazo

La materia está dividida en **dos partes**, y cada una cierra con su parcial.

Parte 1 – Vectores

#	UNIDAD	QUÉ SE AGREGA
01	Introducción a la informática	El mapa del terreno (<i>estamos acá</i>)
02	Entorno y primeros pasos	Node.js, VSCode, terminal, primer programa
03	Variables y operadores	Guardar datos y operar con ellos
04	Condicionales	Que el programa decida
05	Bucles	Que el programa repita
06	Funciones	Dividir el problema en piezas reutilizables
07	Arrays unidimensionales	Manejar muchos datos con un solo nombre

Cierra con el 1er parcial.

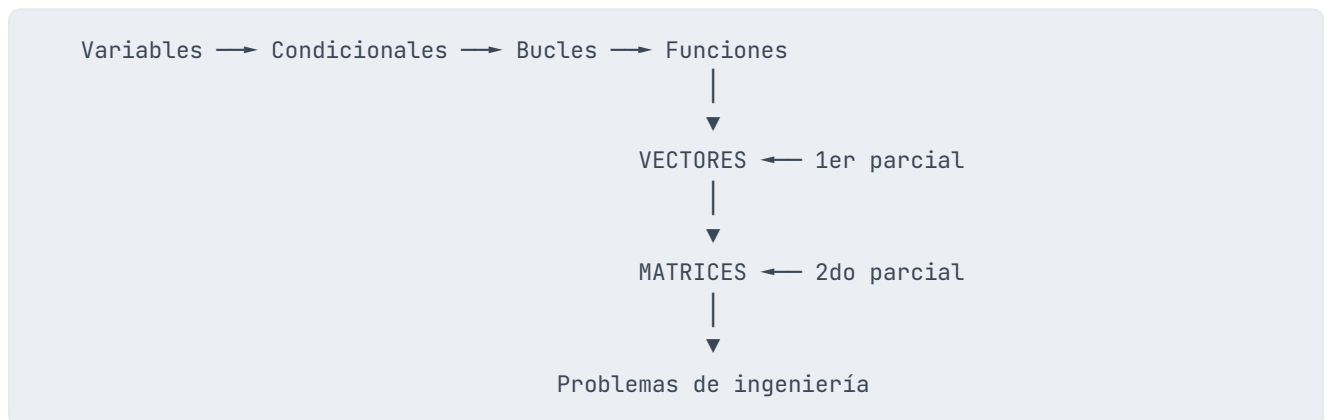
Parte 2 – Matrices

#	UNIDAD	QUÉ SE AGREGA
08	Matrices (arrays 2D)	Datos en filas y columnas
09	Operaciones matriciales	Transpuesta, simetría, producto matricial
10	Integrador de ingeniería	Un problema real de punta a punta

Cierra con el 2do parcial.

Es más corta en unidades pero más densa: todo lo de la parte 1 se usa adentro de dos bucles anidados.

El orden no es negociable



No hay temas independientes. Si te quedás con bucles, vas a arrastrar el problema hasta matrices. Por eso hay un **TP por unidad**: el objetivo es detectar a tiempo lo que no quedó claro, no acumular deuda hasta el parcial.

Las fechas de cada unidad están en el [cronograma](https://github.com/italijancic/pc-2026/blob/main/CRONOGRAMA.md) (<https://github.com/italijancic/pc-2026/blob/main/CRONOGRAMA.md>).

Cómo se evalúa

INSTANCIA	CUÁNDO	QUÉ EVALÚA
TP por unidad	Cada clase	Que puedas resolver solo lo que vimos juntos
1er parcial	Al cerrar la parte 1	Vectores: unidades 01 a 07
2do parcial	Al cerrar la parte 2	Matrices: unidades 08 a 10
Recuperatorio	Al final de la cursada	El parcial que quedó pendiente
Final	En las mesas de examen	Un problema integrador

Nota mínima de aprobación: 6 / 10 (equivale al 60 % del examen correcto).

Cómo es un parcial

Un **problema de ingeniería** partido en **5 incisos progresivos**: el inciso 1 genera los datos y los siguientes calculan métricas sobre ellos.

Ejemplos de parciales ya tomados: simulación de un **ascensor inteligente**, análisis de producción de **paneles solares**, monitoreo de **sensores en una planta industrial**, planificación de **mantenimiento preventivo**. Son problemas de la carrera, no ejercicios abstractos.

Se rinde **escribiendo código en la computadora**, no en papel. Si resolviste los TP de cada unidad, el parcial no trae sorpresas.

Qué se corrige

No sólo que el resultado sea correcto. También:

- **Código modular**: funciones con parámetros y valor de retorno, no todo en el programa principal
- **Nombres descriptivos** para variables y funciones
- **Comentario de cabecera** en cada función
- **Respeto de las restricciones** de la cátedra (ver abajo)

Las reglas de código

PERMITIDO	NO PERMITIDO
<code>for</code> , <code>while</code> , <code>do-while</code>	<code>map</code> , <code>filter</code> , <code>reduce</code> , <code>forEach</code>
<code>if</code> / <code>else if</code> / <code>else</code> , <code>switch-case</code>	<code>find</code> , <code>sort</code> , <code>flat</code> , <code>indexOf</code> , <code>splice</code>
Acceso por índice: <code>v[i]</code> , <code>m[i][j]</code>	<code>fill()</code> y <code>push()</code> para inicializar
<code>.length</code> y <code>.slice()</code>	Retornar objetos o JSON desde una función

Esos métodos existen y son útiles. Pero **resuelven por vos justo lo que tenés que aprender**: la lógica del recorrido. Los vas a poder usar toda tu vida profesional; en esta materia, primero

demostrarás que entendés lo que hay abajo.

1. 🧠 Hardware: las partes físicas

Hardware es todo lo que podés tocar: los componentes físicos de la computadora.

1.1 CPU – Unidad Central de Procesamiento

Es el **procesador**: la parte que ejecuta las instrucciones. Todo lo que hace la computadora pasa, en algún momento, por acá.

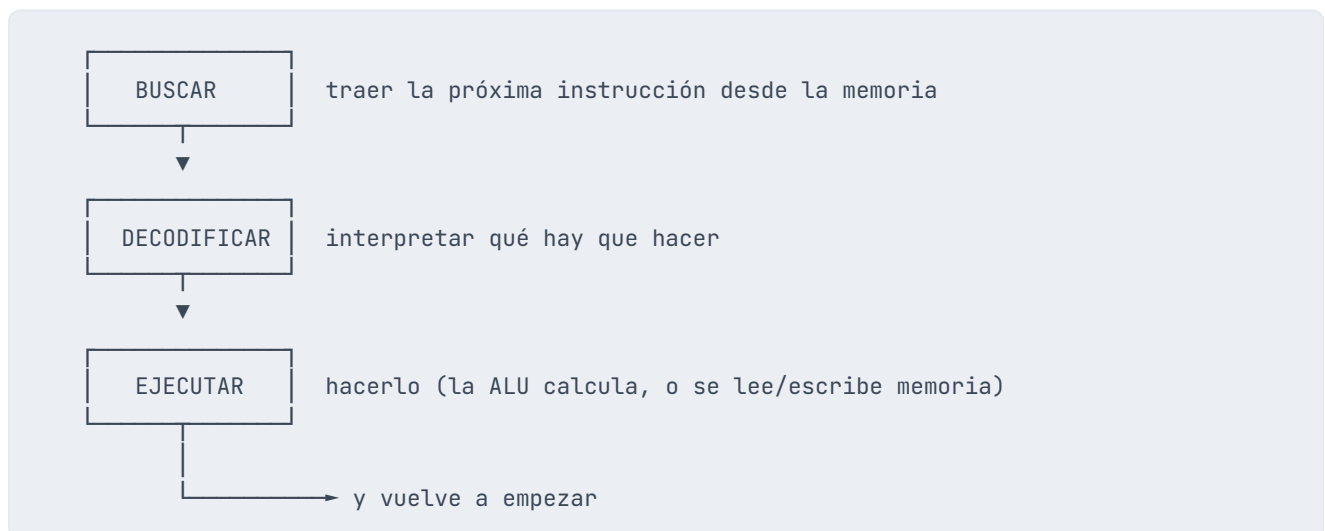
Adentro tiene dos bloques que nos interesan:

BLOQUE	QUÉ HACE
Unidad de Control (UC)	Decide qué instrucción se ejecuta y coordina al resto
Unidad Aritmético-Lógica (ALU)	Hace las cuentas (+ , - , * , /) y las comparaciones (> , < , ==)

Cuando escribas `if (a > b)`, esa comparación la resuelve **la ALU**. Cuando escribas `suma = suma + v[i]`, esa suma también.

1.2 El ciclo de instrucción

La CPU no hace nada sofisticado: repite tres pasos, miles de millones de veces por segundo.



Esto explica los bucles infinitos. Cuando un programa "se cuelga", la CPU no está detenida: está ejecutando tu bucle a toda velocidad, para siempre, porque la condición de corte nunca se hace falsa.

1.3 Memoria: la distinción que más importa

	RAM	ALMACENAMIENTO (SSD / HDD)
Nombre	Memoria de Acceso Aleatorio	Disco de estado sólido / disco rígido
Volátil	Sí: se borra al apagar	No: persiste
Velocidad	Muy rápida	Lenta comparada con la RAM
Capacidad típica	8 – 32 GB	256 GB – 2 TB
Qué guarda	Los programas mientras se ejecutan y sus datos	Archivos, programas instalados, el sistema operativo

⚠ **Esta es la distinción clave de la unidad.** Tus variables viven en la **RAM**. Cuando el programa termina, **desaparecen**. Si querés que algo sobreviva, tiene que escribirse a un archivo, o sea al almacenamiento. Por eso, cuando en un programa cargás un vector de mediciones y el programa termina, las mediciones se fueron. No es un error: es cómo funciona la memoria volátil.

También existe la **ROM** (Memoria de Sólo Lectura): no volátil y no modificable en uso normal. Guarda el *firmware*, el programita mínimo que arranca la máquina y sabe cómo encontrar el sistema operativo.

1.4 Jerarquía de memoria

Hay una regla que se repite en toda la computación: **más rápido = más chico y más caro**.

más rápida	Registros de la CPU	bytes
▲	Caché (L1, L2, L3)	KB – MB
	RAM	GB
▼	SSD / HDD	GB – TB
más lenta	Red / la nube	ilimitado

1.5 El resto de los componentes

COMPONENTE	QUÉ HACE
Motherboard (placa madre)	Conecta todo. Los datos viajan entre componentes por sus <i>buses</i>
GPU	Procesador dedicado a gráficos. Hace muchas operaciones simples en paralelo
Fuente de alimentación	Convierte la tensión de línea en las tensiones continuas que usan los componentes
Periféricos de entrada	Teclado, mouse, sensores: le dan datos a la computadora
Periféricos de salida	Monitor, impresora, actuadores: muestran o actúan con los resultados

💡 **Para tu carrera:** un PLC o un microcontrolador tiene exactamente esta misma estructura —CPU, memoria, entradas y salidas— pero en un encapsulado chico y pensado para leer sensores y accionar motores. Lo que aprendas acá se traslada directo.

1.6 Cómo interactúan

Un ejemplo concreto: **abrís un archivo y lo modificás.**

1. Le pedís al sistema que abra el archivo → se **copia del SSD a la RAM**
2. La CPU lee esa copia desde la RAM y la procesa
3. Escribís cambios → se modifican **en la RAM**, no en el disco
4. Le das *Guardar* → recién ahí la copia de la RAM **se escribe al SSD**

Si se corta la luz entre el paso 3 y el 4, perdiste los cambios. Eso es exactamente lo que significa "guardar".

2. 📁 Software: las instrucciones

Software es todo lo que **no** podés tocar: los programas. Se organiza en capas, y cada capa se apoya en la de abajo.

Software de aplicación	VSCode, Chrome, tu programa
Sistema operativo (OS)	Windows, macOS, Linux
Drivers (software de control)	traducen para cada hardware
Hardware	CPU, RAM, disco, periféricos

2.1 Sistema operativo

Es el programa que **administra los recursos** de la máquina y le da servicios a los demás programas. Sin él, cada programa tendría que saber hablar con cada modelo de disco y de placa de video que existe.

FUNCIÓN	QUÉ RESUELVE
Gestión de procesos	Reparte el tiempo de CPU entre los programas abiertos
Gestión de memoria	Le asigna RAM a cada programa y evita que se pisen entre sí
Gestión de archivos	Crear, leer, escribir y borrar archivos; carpetas y permisos
Gestión de dispositivos	Coordinar la comunicación con los periféricos

Ejemplos: **Windows, macOS, Linux**, Android, iOS.

💡 **Por qué te importa:** cuando tu programa "no encuentra el archivo", casi siempre es porque le estás dando una ruta que el sistema operativo interpreta distinto a lo que imaginás. Vamos a ver eso en la unidad 02, con la línea de comandos.

2.2 Drivers

Un **driver** (o *controlador*) es el software que le permite al sistema operativo hablar con una pieza de hardware específica. Traduce órdenes genéricas —"imprimí esto"— a los comandos puntuales que entiende **ese** modelo de impresora.

Es la razón por la que el mismo Windows funciona con miles de impresoras distintas: el sistema operativo no las conoce, conoce a los drivers.

2.3 Software de aplicación

Los programas que resuelven tareas concretas del usuario: navegadores, planillas de cálculo, CAD, y **los programas que vas a escribir en esta materia**.

2.4 Programa vs. proceso

Dos palabras que se confunden y conviene separar ya:

	PROGRAMA	PROCESO
Qué es	El archivo con las instrucciones	El programa en ejecución
Dónde vive	En el disco	En la RAM
Cuántos puede haber	Uno	Varios del mismo programa a la vez

Tu archivo `app.js` es un programa. Cuando corrés `node app.js`, el sistema operativo crea un **proceso**. Cuando el proceso termina, su memoria se libera.

3. Del código a la máquina: compilados e interpretados

Vos escribís **texto**:

```
console.log('Hola mundo')
```

La CPU no entiende eso. Ella sólo ejecuta instrucciones en **código máquina**: números binarios. Así que en algún momento, alguien tiene que traducir.

🔪 **Siempre hay un traductor.** Lo que cambia entre un lenguaje y otro es **cuándo** traduce.

3.1 Lenguajes compilados

Se traduce **todo el programa de una sola vez**, antes de ejecutarlo. Un programa llamado **compilador** lee tu código fuente y produce un archivo ejecutable en código máquina.

```
codigo.c → COMPILADOR → programa.exe → se ejecuta
           (una vez)      (código máquina)
```

Ese ejecutable ya está en el idioma de la CPU: se corre directo, sin intermediarios.

Ejemplos: C, C++, Rust, Go, Pascal, Fortran.

3.2 Lenguajes interpretados

No hay traducción previa. Un programa llamado **intérprete** lee tu código y lo va ejecutando **a medida que corre**, línea por línea.

```
app.js → INTÉRPRETE → se ejecuta
        (cada vez que corre)
```

No se genera ningún `.exe`: el archivo fuente **es** lo que se distribuye y se ejecuta.

Ejemplos: JavaScript, Python, PHP, Ruby.

3.3 Las diferencias

	COMPILADO	INTERPRETADO
Cuándo traduce	Una vez, antes de ejecutar	Cada vez que se ejecuta
Velocidad de ejecución	Más rápido	Más lento
Errores de sintaxis	Aparecen al compilar	Aparecen al ejecutar esa línea
Portabilidad	Hay que recompilar para cada sistema	El mismo archivo corre en todos
Para probar un cambio	Recompilar y ejecutar	Guardar y ejecutar
Qué se distribuye	El ejecutable	El código fuente

La diferencia en errores es la que más te va a afectar este cuatrimestre: en un lenguaje compilado, un error de tipeo en la línea 50 lo detecta el compilador **antes** de ejecutar nada. En JavaScript, si esa línea nunca se ejecuta, el error **no aparece**. Y si se ejecuta, aparece recién ahí, con el programa a medio correr.

3.4 Cuándo conviene cada uno

Compilado, cuando la velocidad manda: sistemas embebidos, control en tiempo real, drivers, motores de simulación. El PLC de una planta corre código compilado, porque tiene que responder en milisegundos y de forma predecible.

Interpretado, cuando manda la velocidad de desarrollo: automatizar cálculos, procesar datos de mediciones, scripts de ingeniería, aplicaciones web. Un script que analiza el registro de un sensor no necesita microsegundos; necesita estar listo hoy.

3.5 JavaScript es interpretado

Su intérprete es **Node.js**. Por eso lo instalaste: sin él, tu archivo `.js` es sólo un archivo de texto.

De ahí sale una ventaja concreta para la cursada: **el mismo `app.js` corre igual en Windows, macOS y Linux**. Lo que cambia es Node, no tu código. Podés trabajar en la máquina que tengas.

⚠ **Java y JavaScript son lenguajes distintos**, sin relación entre sí más allá del nombre —que fue una decisión de marketing de 1995—. Java compila a un formato intermedio (*bytecode*) que ejecuta la máquina virtual de Java (JVM). Es un caso híbrido entre los dos modelos.

💡 **La realidad es más matizada.** Los motores modernos de JavaScript compilan **sobre la marcha** las partes del código que más se repiten, para acelerarlas: se llama *compilación JIT* (Just In Time). La división compilado / interpretado sigue siendo útil para entender qué pasa, pero en la práctica los lenguajes combinan las dos estrategias.

4. 🛠 Qué pasa cuando corrés un programa

Esto es lo que vas a hacer decenas de veces por clase durante todo el cuatrimestre. Vale la pena saber qué ocurre.

Escribís en la terminal:

```
node app.js
```

Y pasa esto:

1. **La terminal** le pide al sistema operativo que ejecute el programa `node`
2. **El sistema operativo** busca `node` en el disco, lo carga en la **RAM** y crea un proceso
3. `node` abre tu archivo `app.js` y lo lee —para él, tu código es un dato de entrada
4. `node` **traduce** tus instrucciones de JavaScript a instrucciones que la CPU entiende
5. **La CPU ejecuta** esas instrucciones con su ciclo buscar → decodificar → ejecutar
5. Cuando tu código dice `console.log(...)`, `node` le pide al sistema operativo que escriba ese texto en la terminal
7. Al terminar, el proceso muere y **toda su memoria se libera**

📌 **Lo importante del punto 4:** JavaScript no lo ejecuta la CPU directamente. Lo ejecuta **Node.js**, que hace de intérprete. Por eso hay que instalar Node antes de poder correr un programa `.js`, y por eso el mismo archivo `.js` funciona igual en Windows, macOS y Linux: lo que cambia es Node, no tu código.

Eso último es una ventaja concreta para la cursada: podés trabajar en la máquina que tengas.

5. Cómo representa la información: el sistema binario

La computadora sólo maneja **dos estados**: hay tensión o no hay tensión. Eso es un **bit** (**0** o **1**). Todo lo demás se construye a partir de ahí.

5.1 Repaso: cómo funciona el decimal

Ya sabés esto, aunque probablemente nunca lo pensaste así. En el sistema **decimal** (base 10) hay diez dígitos disponibles, del **0** al **9**, y **cada posición vale una potencia de 10**.

Tomemos el número **2026**:

posición:	10^3	10^2	10^1	10^0
	(1000)	(100)	(10)	(1)
dígito:	2	0	2	6
$2 \times 1000 + 0 \times 100 + 2 \times 10 + 6 \times 1 = 2026$				

El valor de cada dígito depende de **dónde está parado**. Es lo que llamamos *notación posicional*.

5.2 El binario es exactamente lo mismo, con dos dígitos

En el sistema **binario** (base 2) sólo hay dos dígitos, **0** y **1**, y **cada posición vale una potencia de 2**.

Tomemos el número binario **1011**:

posición:	2^3	2^2	2^1	2^0
	(8)	(4)	(2)	(1)
dígito:	1	0	1	1
$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$				

1011 en binario es 11 en decimal. No son dos números distintos: es el **mismo número** escrito de dos formas, igual que **0.5** y **1/2**.

Las primeras potencias de 2, que conviene tener a mano:

POTENCIA	2^0	2^1	2^2	2^3	2^4	2^5	2^6	2^7	2^8
Vale	1	2	4	8	16	32	64	128	256

5.3 De binario a decimal

Multiplicá cada dígito por el valor de su posición y sumá. Ejemplo con **110**:

$$1 \times 4 + 1 \times 2 + 0 \times 1 = 6$$

5.4 De decimal a binario

Se divide por 2 repetidamente, se anotan los **restos**, y se leen **de abajo hacia arriba**.

Ejemplo con el **13**:

DIVISIÓN	COCIENTE	RESTO
13 ÷ 2	6	1
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1

Leyendo los restos de abajo hacia arriba: **13 = 1101**.

Verificación: $1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1 = 13$ 

5.5 De ahí salen el 256 y el 1024

Un **byte** son 8 bits. Cada bit puede tomar 2 valores y son independientes entre sí, así que la cantidad de combinaciones posibles es:

$$2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 2^8 = 256$$

Con 8 bits se representan los valores de **0 a 255**: 256 combinaciones. Alcanza para todas las letras del alfabeto, los dígitos y los signos de puntuación.

UNIDAD	EQUIVALE A	POR QUÉ
1 bit	un 0 o un 1	—
1 byte	8 bits	$2^8 = 256$ valores posibles
1 KB (kilobyte)	1024 bytes	2^{10}
1 MB	1024 KB	2^{20} bytes
1 GB	1024 MB	2^{30} bytes
1 TB	1024 GB	2^{40} bytes

 **No son 1000 sino 1024.** La informática cuenta en potencias de 2, no de 10. Por eso un pendrive que dice "16 GB" en el envase muestra menos capacidad en la computadora: el fabricante contó 16×1000^3 bytes y el sistema operativo los divide por 1024^3 .

Con combinaciones de bits se representa **todo**: números, texto, imágenes, sonido.

💡 Esto tiene una consecuencia práctica que vas a encontrar: los números decimales se guardan con **precisión finita**. En JavaScript, `0.1 + 0.2` **no da exactamente** `0.3`. No es un bug del lenguaje: es que `0.1` no tiene una representación exacta en binario, igual que `1/3` no la tiene en decimal. Lo retomamos en la unidad 03.

6. 🐛 Confusiones comunes

CONFUSIÓN	CÓMO ES EN REALIDAD
"La memoria es el disco"	RAM y almacenamiento son cosas distintas. La RAM se borra al apagar
"El programa se colgó, la CPU se detuvo"	Al contrario: la CPU está ejecutando tu bucle a toda velocidad, sin condición de corte
"Guardar y tener abierto es lo mismo"	Tener abierto = está en la RAM. Guardar = se escribió al disco
"Windows es la computadora"	Windows es un sistema operativo. El hardware es el mismo con Linux
"Más GB de disco hace la máquina más rápida"	La velocidad depende sobre todo de CPU y RAM ; el disco es capacidad
"JavaScript lo ejecuta la computadora"	Lo ejecuta Node.js , que traduce para la CPU
"Java y JavaScript son lo mismo"	Son lenguajes distintos . Comparten el nombre por marketing, nada más
"1 KB son 1000 bytes"	Son 1024 — la informática cuenta en potencias de 2
"El binario es otro número"	Es el mismo número escrito en otra base. <code>1011</code> y <code>11</code> son el mismo valor

7. Resumen

CONCEPTO	EN UNA LÍNEA
Hardware	Las partes físicas: CPU, memoria, almacenamiento, periféricos
CPU	Ejecuta instrucciones. Contiene la Unidad de Control y la ALU
Ciclo de instrucción	Buscar → decodificar → ejecutar, repetido sin parar
RAM	Memoria de trabajo, rápida y volátil . Aquí viven tus variables
Almacenamiento	SSD / HDD: lento y persistente . Aquí viven tus archivos
Software	Los programas. Se organizan en capas
Sistema operativo	Administra procesos, memoria, archivos y dispositivos
Driver	Traduce entre el sistema operativo y un hardware específico
Programa	El archivo con instrucciones, en el disco
Proceso	El programa ejecutándose, en la RAM
Compilado	Se traduce entero antes de ejecutar. Produce un ejecutable. C, C++, Rust
Interpretado	Se traduce mientras se ejecuta. JavaScript, Python, PHP
Node.js	El intérprete que ejecuta tu JavaScript
Binario	Base 2: cada posición vale una potencia de 2
Bit / byte	1 bit = 0 o 1 . 1 byte = 8 bits = 2^8 = 256 valores

8. Para la próxima clase

En la **unidad 02** instalamos el entorno de trabajo y corrés tu primer programa. Para aprovechar la clase, llegá con esto hecho:

1. **Node.js** instalado — versión **LTS** — desde <https://nodejs.org/en/download>
(<https://nodejs.org/en/download>)
2. **Visual Studio Code** instalado — desde <https://code.visualstudio.com/>
(<https://code.visualstudio.com/>)
3. Verificado que funcionan: abrí una terminal y ejecuté

```
node -v
```

Si responde algo como **v24.19.0**, está listo.

Si algo falla, **no te quedes trabado**: traé el error anotado o sacale una foto a la pantalla y lo resolvemos en clase. Es normal que la instalación tenga vueltas, sobre todo en Windows.

Material de la unidad

- [Presentación de clase](https://github.com/italijancic/pc-2026/blob/main/unidades/01-intro-informatica/presentacion.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/01-intro-informatica/presentacion.pdf)
- [Trabajo Práctico](https://github.com/italijancic/pc-2026/blob/main/unidades/01-intro-informatica/tp.pdf) (https://github.com/italijancic/pc-2026/blob/main/unidades/01-intro-informatica/tp.pdf)